DP Computer Science: A 1.1.6 Multicore Architectures & Pipelining (HL Only)

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of the lesson, HL students will be able to:

1. Name, in order, the **four stages of the instruction pipeline** (fetch, decode, execute, write-back) and state what happens at each
2. Describe how **overlapping the stages** lets one core hold several instructions in progress at once, raising **throughput** without reducing the **latency** of any single instruction
3. Describe how each core of a **multi-core processor** runs its own pipeline, so the cores work independently and in parallel on tasks assigned by the operating system
4. Describe **pipelining** and **parallel processing** as two distinct levels of concurrency and keep them apart in an answer

> **Command Term Note:** The lesson is held at **Describe**—a detailed, ordered account. Speedup calculations and pipeline hazards have been removed; the cycle-count grid remains but to show overlap, not to be solved.

Key Vocabulary (Tier 3)

Term	Definition
Pipelining	Splitting the execution of an instruction into separate stages so the stages of consecutive instructions can overlap inside one core
Write-back	The final pipeline stage, where the result of the operation is stored in a register or in memory for later use
Throughput	The number of instructions completed per unit of time; **this is what pipelining improves**
Latency	The time to complete one whole instruction from start to finish; **pipelining does not reduce this**
Core	A complete processing unit with its own pipeline, ALU, and registers, able to run its own instruction stream
Parallel processing	Different cores carrying out different tasks at the same instant, coordinated by the operating system

---

Etymology Notes (for the Board)

Term	Origin / Meaning
Pipeline	Work flows through and no section sits empty
Concurrent	Latin *currere* — "to run" — so "running together"
Parallel	Side by side, at the same time

Pedagogical Focus

- **Communication Skills (ATL):** Students consciously practise communicating a technical process precisely, in the correct sequence, using Tier 3 terms accurately. Name it at the start: "*Today you are practising describing a process so a reader could follow it exactly.*"
- **Differentiation (ATT):** Vocabulary is treated as the main barrier to a correct Describe answer. Tier 3 terms are taught explicitly and rehearsed before any extended writing.

Secondary ATLs present: Thinking skills (transferring the single-cycle model from A1.1.5 to the overlapped case) and self-management (organising a multi-stage description in order).

EAL Scaffold

Support Type	Description
Sentence Frame	"Pipelining splits the execution of an instruction into four stages: fetch, decode, execute, and write-back. Because each stage uses _____ hardware, a core can _____ the stages of consecutive instructions, so several instructions are _____ at once. This raises _____ without reducing the _____ of any one instruction. In a multi-core processor, each core has its own _____, so the cores work _____ and in _____ on different tasks assigned by the _____."
Idiom Flag	The laundry analogy assumes washing machine → tumble dryer → iron as a known sequence. Offer a plain alternative: any three-step line where each station does one job, such as a drive-through car wash (wash, rinse, dry bays).
Metaphor Flag	"Staircase pattern" → Plain version: each instruction starts one stage later than the one above it, so the shaded boxes step down and to the right.
Compound Word Flag	"Write-back" may be misread; say it means writing the result back into a register or memory.

---

2. Lesson Sequence (One-Hour Block)

Part 1: Do Now (Retrieval) — ~10%

Activity: Four short recall items from prior learning:

Question	Source
1. Name the three stages of the basic instruction cycle and say what happens at fetch	[A1.1.5]
2. Which register holds the address of the next instruction to be fetched?	[A1.1.5]
3. Name the part of the CPU that carries out arithmetic and logic operations	[A1.1.1]
4. In one sentence, what is the role of the control unit during the instruction cycle?	[A1.1.5]

Part 2: Main Sequence — ~60%

Phase A: Pre-Teach Vocabulary from a Concrete Example (~15 mins)

Step 1 — The Laundry/Lines Analogy (Introduce Overlap):

Method	Sequential (No Pipeline)	Pipelined (With Overlap)
1 Load	Wash (30m) → Dry (30m) → Fold (30m) = **90m**	Same = 90m
3 Loads	3 × 90m = **270m**	~**150m** (overlapping stages)

Key Takeaway:

- **Latency** (time for one load) didn't change
- **Throughput** (loads per hour) increased

Step 2 — Map onto the Four Stages:

Stage	What Happens
Fetch	Instruction is retrieved from memory
Decode	Instruction is split into opcode and operand; control signals generated
Execute	Operation is performed (ALU)
Write-back	Result is stored in register or memory **(new fourth stage)**

> **Recall:** Fetch, decode, execute from A1.1.5; **write-back** is added as the new fourth stage.

Step 3 — Frayer Model: Throughput vs. Latency:

	Throughput	Latency
Definition	Instructions completed per unit of time	Time to complete one instruction
Pipelining Effect	**Increases**	**Does not reduce**
Example	100 instructions per second	10 nanoseconds per instruction

Step 4 — Example/Non-Example: Pipelining vs. Parallel Processing:

	Pipelining	Parallel Processing
Example	One core with overlapping stages	Multiple cores working simultaneously
Non-Example	Multiple cores (that's parallel, not pipelining)	One core (that's pipelining, not parallel)
Key Distinction	**Within a single core**	**Across multiple cores**

> **The non-example matters most:** Pipelining does **not** make a single instruction faster.

Phase B: Reading with a Purpose (~10 mins)

Task: Students read the eBook A1.1.6 sections on:

1. The four stages of pipelining
2. Pipelining in multi-core architectures
3. How cores work independently and in parallel

Worksheet Part 1 — Pipeline Grid:

- Students shade the grid so each instruction steps down one stage per row
- The boxes should form a **diagonal "staircase" pattern**

Clock Cycle	Instruction 1	Instruction 2	Instruction 3
1	**F**		
2	D	**F**	
3	E	D	**F**
4	W	E	D
5		W	E
6			W

Two-Row Table — Distinguishing Levels of Concurrency:

	Within-Core Pipelining	**Across-Core Parallel Processing**
What it is	Overlapping stages of multiple instructions in one core	Different cores running different tasks simultaneously
What it improves	Throughput	Throughput (via task division)
Coordination	Hardware-controlled	Operating system-controlled

Phase C: Oracy Bridge (~5 mins)

Activity (Pairs):

- Each student explains **one level of concurrency** to the other using the terms
- The listener checks:
  - Are the four stages named in order?
  - Is "independently and in parallel" used correctly?

> **Talk first, write second.**

Part 3: Deliberate Practice — ~20%

Task: Independent written account of the model question:

> "*Describe how pipelining in a multi-core processor improves overall system performance.*"

Sentence Frame (removable scaffold):

> *"Pipelining splits the execution of an instruction into four stages: fetch, decode, execute, and write-back. Because each stage uses _____ hardware, a core can _____ the stages of consecutive instructions, so several instructions are _____ at once. This raises _____ without reducing the _____ of any one instruction. In a multi-core processor, each core has its own _____, so the cores work _____ and in _____ on different tasks assigned by the _____."*

Outcome: Every student produces the same Describe-level target.

Part 4: Exit Check — ~10%

Two-Part Check:

Question	Purpose
Part A: "Outline what happens at the decode and write-back stages" **[2 marks]**	Checks understanding of specific stages
Part B: Self-check: Did I name all four stages in order, and did I say each core runs its own pipeline independently and in parallel?	Promotes metacognition

Gaps to Watch For:

Error	Misconception
"Pipelining makes one instruction faster"	Latency confusion
Describing only one level of concurrency	Missing the distinction between within-core and across-core
Missing write-back	Not adding the fourth stage

---

3. Visual Summary: Pipelining vs. No Pipelining

No Pipelining (Sequential)

```
text

Instruction 1:  F → D → E → W
Instruction 2:                  F → D → E → W
Instruction 3:                                  F → D → E → W
```

Total time for 3 instructions: 12 clock cycles

With Pipelining (Overlap)

```
text

Instruction 1:  F → D → E → W
Instruction 2:      F → D → E → W
Instruction 3:          F → D → E → W
```

Total time for 3 instructions: 6 clock cycles

> **Key Insight:** Throughput increases; latency of each instruction remains 4 cycles.

---

4. Differentiation & Extension

Support (Scaffolds That Can Be Removed)

Support Strategy	Description
Sentence Frame	Provided for written account; withdrawn for second attempt
Stage Cards	Cut-up set of four stage cards to sequence before writing

Support Strategy	Description
Frayer Sheets	Left visible during initial writing
Partially Completed Grid	Pipeline grid with some stages already shaded

Challenge (Deeper or Wider, Held at Describe)

Challenge Strategy	Description
Three-Instruction Example	Describe why three instructions take 6 clock cycles pipelined against 12 unpipelined—a fuller account of why throughput rises while latency is unchanged
Limits of Multi-Core	Describe two limits: (1) a strictly sequential task cannot be split across cores; (2) cores share some resources such as main memory
Stalls/Hazards	Describe why a pipeline sometimes has to wait for data (bubbles/stalls)

> **Keep both at the level of a detailed account.** Do not move into deriving a speedup formula or evaluating hazards.

---

5. Lesson Activities (Supplementary)

If time allows, these can replace or supplement the main sequence:

Phase	Time	Activity
Introduction	0-10 min	The "Laundry" Analogy — sequential vs. pipelined
Deep Dive	10-25 min	F-D-E-W cycle explanation + Worksheet Part 1 (pipeline grid)
The Math	25-40 min	Cycle-count grid showing overlap; introduce hazards briefly
Multi-Core Integration	40-55 min	Quad-core = four pipelines; case study discussion
Wrap Up	55-60 min	Recap + Exit ticket

Case Study Discussion Questions:

- "If I have 4 cores, why does my old game not run 4x faster?" (Software must be written to use the cores)
- "Rendering video vs. running a linear Python script — which benefits more from multiple cores?"

---

6. Learning Objectives (Summary)

Knowledge and Understanding

- Explain the concept of pipelining and how it improves CPU performance
- Describe the four stages of pipelining (fetch, decode, execute, write-back) and their functions
- Explain how pipelining works in multi-core architectures, where each core has its own pipeline
- Describe how cores in multi-core processors operate independently and in parallel

Skills

- Analyse the flow of instructions through a pipeline and identify potential hazards
- Evaluate the impact of pipelining on instruction throughput and execution time
- Explain the advantages of multi-core processors for multitasking and parallel processing

---

7. Resources

- **Presentation** (Slide Deck)
- **InThinking eBook** (A 1.1.6 with quiz)
- **Worksheet:** Pipeline Grid (Part 1) + Table (Part 2)
- **Worksheet Answers**

Author's Reflection

> "*This lesson focuses on differentiation because the Describe statement is held back by vocabulary, not necessarily by the difficulty of the idea. A student who uses the vocabulary 'throughput,' 'latency,' 'pipeline,' and 'core' accurately can write a clean four-mark account; one who cannot will blur them however well the laundry analogy in the lesson lands. So in the lesson the words come first and the writing second to support this.*

> *Holding it at Describe meant a cut from the original lesson. The legacy deck carried a speedup calculation and a run at pipeline hazards, and both have gone. The cycle-count grid stays, but to show the overlap, not to be solved.*

> *The link I cared most about is the handoff from A1.1.5, which stopped at three stages. Here write-back is named as the new fourth stage rather than slipped in, because it is the one students drop once the two levels of concurrency start competing for their attention.*"